COMPUTER ENGINEERING
QUEZON CITY

Final Project: Barangay Document Generator

Course Code: CPE 010 **Course Title: Data Structures and Algorithms**

PROJECT INFORMATION

Item	Details
e-BarangayDocs: A Digital Request System	A system where residents request barangay documents that replaces manual paper forms, long queues and handwritten logbooks.
Business Process Emulated	Barangay Document Request and Queueing
Section / Group No.	CPE21S2 / Team 1
Instructor	Engr. Jimlord M. Quejado
Date Submitted	November 6, 2026
GitHub Repository	https://github.com/Limpiocpe/CPE010_FinalProj_Team1 (details in Appendix C)

Group members, roles, and contributions are listed in **Appendix B. Team Composition**.

Intended Learning Outcomes Addressed

At the end of this activity, the students should be able to:

- Apply fundamental data structures and techniques to solve engineering-related problems through algorithm design.
- Design efficient algorithms and analyze their complexity.
- Build efficient algorithms based on selected data structures using good programming styles in C++.

Objectives

General Objective: To develop a C++ program that emulates the efficiency process of barangay documents using appropriate data structures and algorithms.

Specific Objectives:

1. To manage documents and requests to barangays using a queue so that residents are served in first-come, first-served order.
2. To search documents, and request records efficiently using binary search.
3. To sort accepted documents, and request reports by using merge sort.
4. To analyze the time complexity of the main operations theoretically and experimentally.

1. BUSINESS PROCESS

1.1 Organization and Business Process

The chosen organization is the Barangay Hall that provides services to the residents of a community in which one of their important services is to process barangay documents. In this business process, the persons involved are the residents and the barangay staff such as the barangay captain.

This business process starts by letting the resident upload their concerned document then the staff will check for its requirement and will record its request. After recording its request, the barangay staff will give the resident a special ID / ticket number where the resident will present it upon retrieval of their documents. While processing their document, the barangay staff must update the progress. Once the document is done, it will be submitted for approval then the resident will pay for the document and have their official receipt. Once the payment is done, the resident will give their ticket number to retrieve their documents.

1.2 Current Process (As-Is)

Step	Activity	Performed By	Data Involved
1	Residents may pass, or request documents to the barangay	Resident	Name, address, document type
2	Documents will be queued in the system with the allotted time of completion	Barangay Staff	Document type, document special ID
3	Documents that are being processed will be delivered to the barangay official/captain to be approved or signed	Barangay official/captain	Document type, document special ID
4	Documents will have a final reading before being handed to the specific resident	Barangay Staff	Official Signed Document, Document Special ID

1.3 Problems in the Current Process

1. **Manual Submission of Paper Forms**: Submitting paper forms uses physical resources and reduces accuracy in data collection due to manual transfer of information due to human error, such as missed details and illegible handwriting.
2. **Manual Workflow:** Physical signatures, documentation approvals, and lack of electronic tracking equipment slows down the delivery of documentations.
3. **Usage of Handwritten Logbooks**: Using handwritten logbooks may cause resident records to be lost due to misplacement, stealing, and other physical damages.
4. **Documentation Archives:** Many workplaces still rely on physical paper storage or outdated digital infrastructure. Which causes processing delays due to searching through filing rooms.
5. **Lack of Real-Time Tracking**: Processing of documents may be slow or halted due to lack of real-time tracking. This causes multiple other problems such as long queues and delay in sending the document.
6. **False Priority**: Residents may not be prioritized accordingly by First-In First-Out (FIFO).

1.4 Proposed System

#	Feature	Business Task It Supports
1	Resident login and online request form, with the list of required documents per document type.	Document request submission
2	Search for retrieving requested document records and verify if it exists.	Document record retrieval and verification
3	Automatic ticket number (special ID) when a request is recorded, and addition to the processing line.	Queue management (Queue, 2.2)
4	Updates the request's status to indicate the progression.	Request status tracking
5	Generates reports based on selected criteria, such as date, ticket ID, or document status.	Report generation
6	Main Menu; Allows users to select a system function or exit the application.	System navigation and control

1.5 Scope and Delimitations

- The system covers only a digital submission of a requested document.
- The system covers a queue that is based on First-In, First-Out and a ticket number will be given.
- The system covers searching request records using a binary search.
- The system displays the status history of the request, including its status, date and time of change, and staff member in charge.
- The system does not accept an actual online payment for the documents.
- The system does not automatically approve the document signed by the officials.
- The system does not use an online database where records and documents will be stored.

2. DATA STRUCTURES AND ALGORITHMS APPLIED

Course topics you may choose from: arrays and dynamic memory allocation; singly, doubly, and circular linked lists; stacks (array or linked-list based); queues (array or linked-list based); linear search; binary search; hashing; sorting algorithms; general trees and binary search trees; AVL trees; tree traversal algorithms; graphs (adjacency matrix or adjacency list); breadth-first search; depth-first search.

2.1 Summary of Data Structures and Algorithms Applied

#	Data Structure / Algorithm	Where It Is Used in the Business Process	Main Operations
1	Queue / FIFO	Residents' requests waiting for the clerk to verify them in order	enqueue, dequeue, peek

2	Merge Sort	Sorting released requests by data, document type or fee amount	Split, merge, compare
3	Linked List	Request's status history	Inverse at end, traverse, search
4	Binary Search	Finding a special id / date range in release logbook	Binary search, lower and upper bound
5	Stack / LIFO	Undoing any human error	Push, pop

2.2 Queue (FIFO)

Concept: A queue is a line of items where new items join at the rear end, and the items leave from the front or first-in first-out basis.

Application in the System: Holds the requests that are waiting to be processed. Each item stores the special ID, resident name, document type, date submitted and the submitted requirements.

Justification: Long lines and skipped requests are one of the main problems of the paper process, so the system must keep a fair and visible order. Queue ensures first come, first served, and does not let a staff pass over an older request.

Operations and Complexity:

Operation	Purpose	Time
enqueue	Add a request at the rear	O(1)
dequeue	Take the next request for processing	O(1)

2.3 Merge Sort

Concept: Merge sort sorts a list by splitting it in half, again and again until every piece has only one item, which is already "sorted". Then slowly, the pieces are merged back together in pairs: the front items of two sorted pieces are compared, the smaller one is taken first, and repeats until both pieces are used up.

Application in the System: The staff needs a list of released requests in a chosen order for reports: by date released, document type or fee amount.

Justification: Other sorting algorithms such as bubble sort takes $O(n^2)$ time, which becomes slow once the barangay has hundreds of released requests.

Operations and Complexity: [Main operations used and their time complexity, e.g., enqueue O(1), search O(log n). Full analysis goes in Section 5.]

2.4 Linked List

Concept: A linked list is a chain of nodes, each node holds some data and a pointer to the next node, and the list remembers where the chain starts (or the head node) and ends (tail node).

Application in the System: Every request has its own linked list that stores its status history. Each node holds the status, the date and time, and the staff member who made the change.

Justification: The number of status changes is not the same for every request: a barangay clearance has fewer steps than a business permit that is returned and resubmitted.

Operations and Complexity: [Main operations used and their time complexity, e.g., enqueue O(1), search O(log n). Full analysis goes in Section 5.]

2.5 Binary Search

Concept: Binary Search works by finding a specific item in a sorted list then dividing it into two halves. The process continues until the target item is found and when there are no more elements to search.

Application in the System: Binary Search is used to find the requested document using the ticket number given by the resident when they retrieve their document.

Justification: Binary Search can search better than Linear Search as this does not check one-by-one instead it is divided into halves-by-halves.

Operations and Complexity: [Main operations used and their time complexity, e.g., enqueue O(1), search O(log n). Full analysis goes in Section 5.]

2.6 Stack (LIFO)

Concept: A stack is a pile where items are added and removed only at the top, so the last item is the first one out or the last-in, first-out basis. Like a stack of plates, you take the top plate only or the one you just placed.

Application in the System: Each staff has an undo stack for the actions made in the current section. Every action is pushed with the old value, the new value, the ticket number and time.

Justification: Undoing must reverse the most recent action first, which is exactly how a stack works.

Operations and Complexity: [Main operations used and their time complexity, e.g., enqueue O(1), search O(log n). Full analysis goes in Section 5.]

3. SYSTEM FLOWCHART

Symbol	Name	Use
Oval	Terminator	Start and End
Rectangle	Process	An action or computation, e.g., "Add patient to queue"
Parallelogram	Input / Output	Reading input or displaying output
Diamond	Decision	A yes/no question, e.g., "Is the queue empty?"

Rectangle with double sides	Predefined Process	An operation detailed in another flowchart
Arrow	Flow Line	The order of steps

3.1 Main System Flowchart

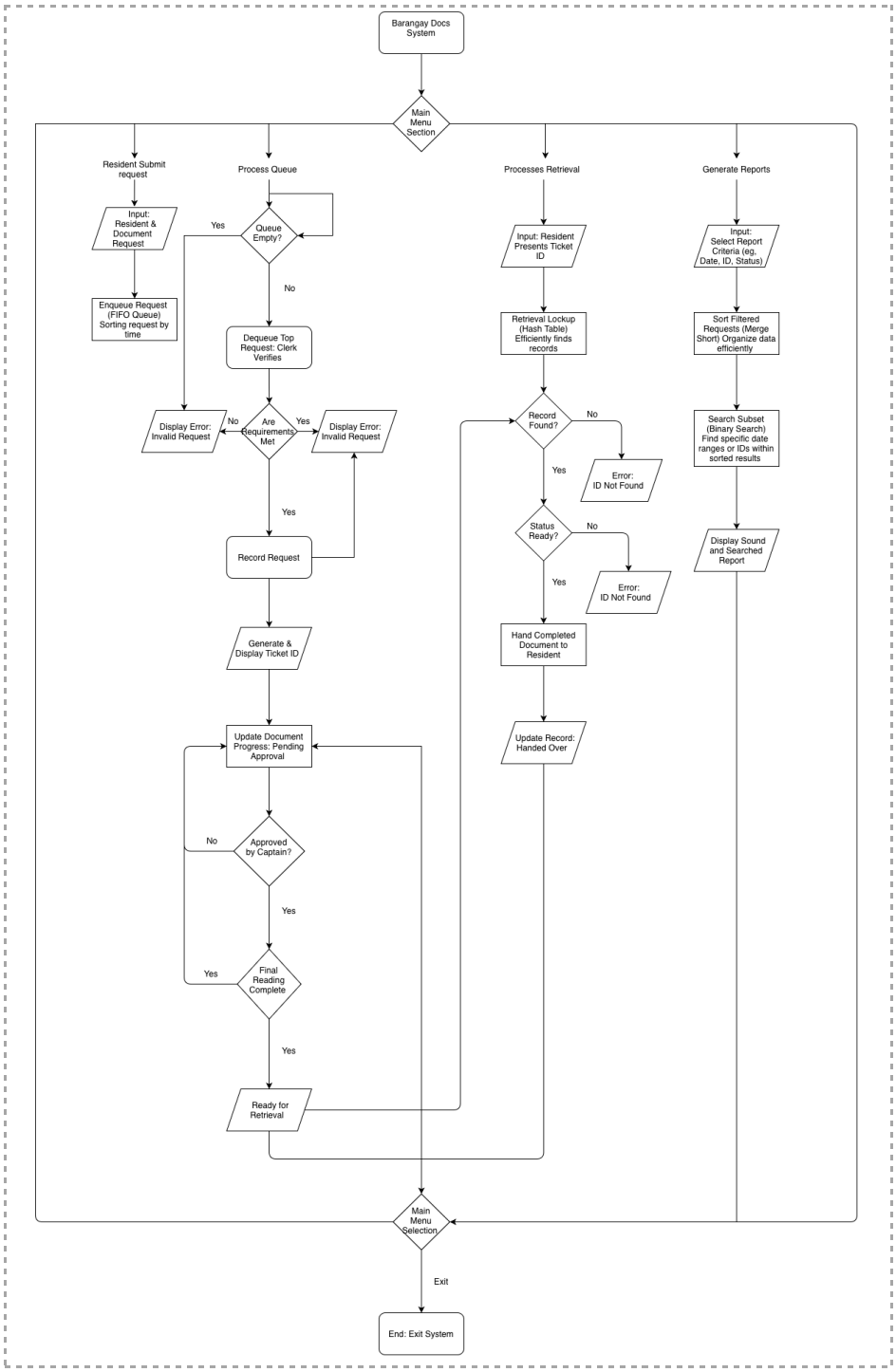


Figure 1. Flowchart of Barangay Document Generator

3.2 Flow Description

1. **Program Initialization (Start)**: The system starts and initializes all necessary data structures, including the request queue, hash tables for user accounts, and storage arrays for document records.

2. **Main Menu Navigation:** The user is presented with the Main Menu to select their role or desired system task:
    - **Resident Submits Request:** Proceeds to the request creation flow.
    - **Clerk Processes Queue:** Navigates to the pending request queue verification.
    - **Clerk Processes Retrieval:** Navigates to the document lookup and release module.
    - **Generate Reports:** Navigates to the administrative search and report generation module.
    - **Exit:** Terminates the program execution.

3. **Branch 1 - Resident Submits Request:**
    - The resident inputs personal details and selects the document type.
    - The system executes an operation to place the request into the **First-In, First-Out(FIFO) Queue,** ordering incoming requests chronologically by submission time.

4. **Branch 2 - Clerk Processes Queue:**

- The system checks if the request queue is empty. If empty, it returns to the main menu selection.
- If non-empty, the clerk performs a dequeue operation to fetch the top pending request.
- **Decision (Requirements Check)**: The clerk verifies if all document requirements are met.
    - If **No**, an "Invalid Request" error is displayed, and the request is rejected.
    - If **Yes**, the clerk records the request, and the system generates a unique **Ticket / Special ID** for the resident.
- The document status is updated to **Pending Approval**.
- **Decision (Captain Approval)**: The Barangay Captain reviews and signs the document.
- **Decision (Final Reading)**: The document undergoes a final review before being marked as **Ready for Retrieval**.

5. **Branch 3 - Clerk Processes Retrieval**
    - The resident presents their assigned Ticket ID.
    - The system performs a Hash Table retrieval lookup to instantly access the residents record in O(1) average time.
    - Decision (Record Found?): if the ID does not exist in the hash table, an “ID Not Found” error is returned.
    - Decision (Status Ready?): if found, the system checks the document to the residents and updates the status to Handed Over.

6. **Branch 4 - Generate Reports**
    - The admin selects report criteria(such as Date Range, Document Type, or Status).

- Merge Sort Execution: The system applies Merge Sort to arrange filtered document records efficiently with an O(n log n) time complexity.
- Binary Search Execution: A Binary Search is executed on the sorted dataset to instantly locate specific ID ranges or dates in O(n log n) logarithmic time.
- The sorted and searched report is displayed on screen for the admin.

4. OUTPUT

Feature	Figure No.	Input Used
Resident login and request form	Figure 2	Menu choice 1 Name: Marc Bungag, Document Type: Building Permit Date: 2026-11-06 Requirements: Valid ID, Approval
Search and verify a document record	Figure 3	Menu choice 5 Ticket Number: BRG-2026-0001
Auto ticket number and Queue and Status update	Figure 4	Four requests submitted in sequence (Marc Bungag, Juan Marques, Marqui de Gramont, John Wick), with the system auto-assigning BRG-2026-0001 to 0004
Main Menu	Figure 5	Four requests submitted in sequence (Marc Bungag, Juan Marques, Marqui de Gramont, John Wick), with the system auto-assigning ID's BRG-2026-0001 to 0004

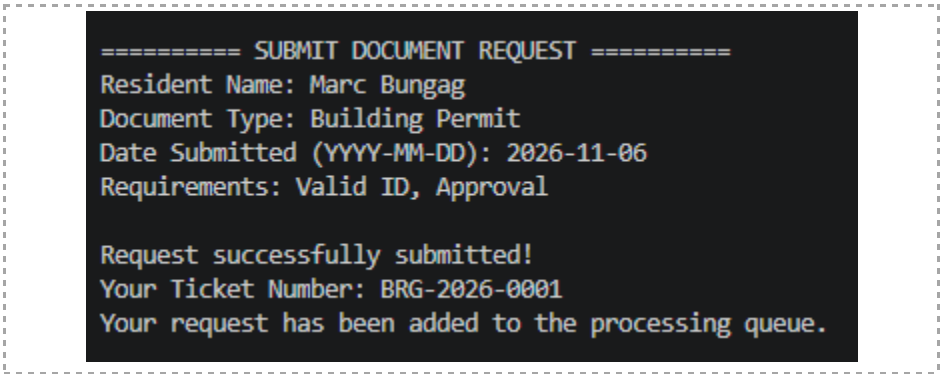


Figure 2. Main menu and Resident Log In

The image shows the main menu of the program wherein it has options that users can select what function they want to use. Under the main menu is where residents can input their name, documents needed, the date, and the requirements for acquiring the certain document. The data is then stored in a queue.

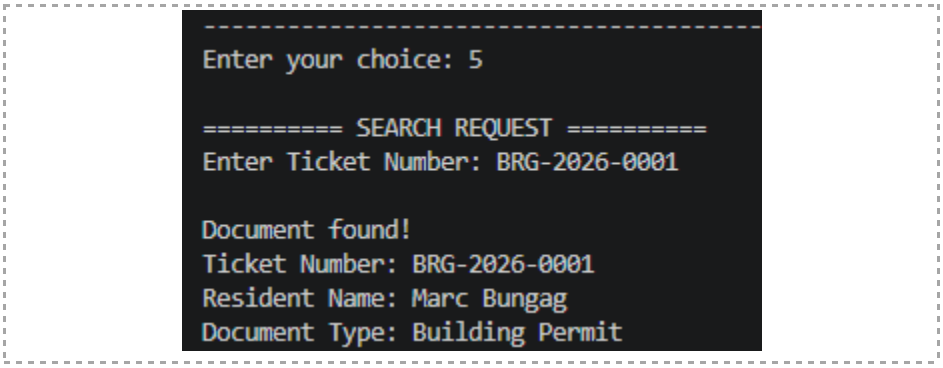


Figure 3. Search and Verify a Document Record

In this part, users can verify and track the document after being processed. The document must be processed first before searching it using a binary search algorithm. Each document has a special ticket number in it, so that each document is unique to each other.

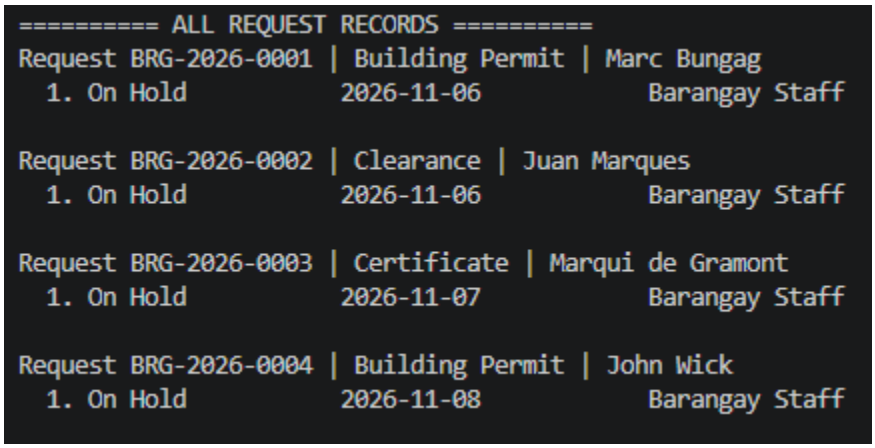


Figure 4. Auto Ticket Number and Queue

The figure shows how the system automatically generates a unique ticket number for every submitted request and organizes them in a queue. Each record displays the request ID, type of request, applicant's name, current status, date, and assigned office. All four requests are marked as "On Hold," making it easier for the Barangay Staff to monitor pending applications. This feature helps prevent duplicate records, keeps requests organized, and allows staff to track and manage applications more efficiently.

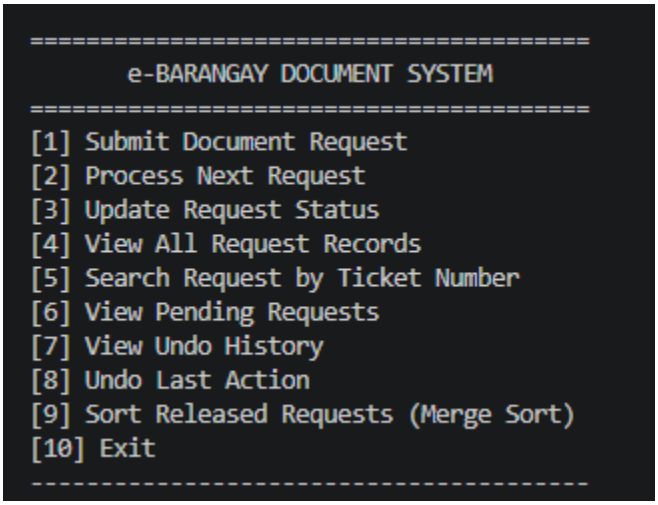


Figure 5: Main Menu

The figure shows the main menu of the e-BARANGAY Document System, which provides users with different options for managing document requests. It allows staff to submit new requests, process the next request, update request statuses, view all records, and search for specific requests using ticket numbers. It also includes features for viewing pending requests, checking undo history, reversing the last action, and sorting released requests using the Merge Sort algorithm. These functions help make document processing more organized, accessible, and efficient while reducing manual work for Barangay Staff.

5. PERFORMANCE ANALYSIS

5.1 Theoretical Analysis

Operation	Data Structure / Algorithm	Best	Average	Worst	Space
Add request	Queue (linked list)	O(1)	O(1)	O(1)	O(1)
Process next request	Queue (dequeue)	O(1)	O(1)	O(1)	O(1)
Search record by special ID	Binary search	O(n)	O(n)	O(n)	O(1)
Update request status	Linked list	O(1)	O(n)	O(n)	O(1)
Generate report (sort by date, ticket ID, or type)	Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Undo last action	Stack (array)	O(1)	O(1)	O(1)	O(1)

Derivation: Based on the theoretical analysis, Queue and Stack operations are the most efficient as they both require O(1) while Merge Sort also provides an efficient way in organizing the released document records as it has a time complexity of O(n log n). On the other hand, the searching and updating records both require O(n) because the

system needs to traverse the linked list to locate a specific record. All of these complexities help explain how the system performs as the number of document requests increases.

5.2 Experimental Analysis

Setup Item	Details
Computer (CPU, RAM)	Intel Core i5-12400F, 16 GB RAM
Operating system	Windows 11
Compiler and settings	g++ 13.2,
Timing method	std::chrono::steady_clock around the operation only
Input data	Randomly generated records
Repetitions	5 runs per input size]

Input Size (n)	Binary Search Time (ms)	Merge Sort Time (ms)	Queue Time (ms)
100	0.0107	0.0186	0.0294
1,000	0.0063	0.1976	0.34600
10,000	0.122	3.0985	4.37610
100,000	3.3685	30.1616	42.2995

```
========== RUNNING TIME RESULTS ==========
Input Size      Binary Search       Merge Sort          Queue Operations
100             0.010700            0.018600            0.029400
1000            0.006300            0.197600            0.346000
10000           0.122000            3.098500            4.376100
100000          3.368500            30.161600           42.299500
```

Figure 5. Running time vs. input size for Binary Search, Merge Sort, and Queue

The results show that the execution time generally increases as the input size becomes larger. Binary Search has the lowest running time among the three operations, reaching 3.3685 ms at 100,000 inputs. Merge Sort takes longer as it processes and organizes more document requests, reaching 30.1616 ms. Queue Operations have the highest execution time, reaching 42.2995 ms because each request must be inserted and removed from the queue. Although Binary Search has a lower running time, its implementation uses a linked list, which requires node

traversal to locate the middle element. The results demonstrate that larger datasets require more processing time, especially for operations that handle multiple records.

5.3 Comparison of Theoretical and Experimental Results

Operation	Expected Growth (Big-O)	Observed Growth	Agrees? (Yes / No)
Binary Search (linked list)	O(n)	about 0.6x, 19x, then 28x	Partly: not logarithmic, and steeper than linear
Merge Sort	O(n log n)	about 10.6x, 15.7x, then 9.7x	Yes
Queue operations (n enqueue/dequeue)	O(n), O(1) each	about 11.8x, 12.6x, then 9.7x	Yes

6. CONCLUSION AND RECOMMENDATIONS

6.1 Conclusion

The Barangay Document Generator successfully emulates the efficiency process of barangay documents using appropriate data structures and algorithms. By applying the FIFO Queue for processing requests, Binary Search for quick record retrieval, Merge Sort for generating a formal document, Linked Lists for updating request status, and a Stack for undoing errors or actions, the system successfully achieves efficient performance.

6.2 Recommendations

- **Reliable Storage via Database** - This will help in preventing the data loss when the program closes.
- **Document Status Notifications** - A feature where the residents will receive a notification regarding their document status.
- **Advanced Report Generation** - A filter option that contains monthly requests, most requested documents, and completed documents.
- **User Authentication and Role Management** - Create a separate account for residents, staff, and administrators to ensure that only authorized individuals can only access it.
- **Priority-Based Request Processing** - An optional priority system for urgent requests while maintaining FIFO for regular requests.

7. REFERENCES

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

cppreference.com. (n.d.). *C++ reference*. https://en.cppreference.com/

Goodrich, M. T., Tamassia, R., & Mount, D. M. (2011). *Data structures and algorithms in C++* (2nd ed.). Wiley.

Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2nd ed.). Addison-Wesley.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.

Weiss, M. A. (2014). *Data structures and algorithm analysis in C++* (4th ed.). Pearson.

APPENDICES

Appendix

A. Source Code

A.1 main.cpp

```
#include <iostream>
#include <string>
#include <limits>
#include <vector>
#include "stack.h"
#include "Queue.h"
#include "LinkedList.h"
#include "BINARY_SEARCH.h"
#include "MergeSort.h"
RequestQueue::~RequestQueue() {
    while (!isEmpty()) {
        DocumentRequest* request = dequeue();
        delete request;
    }
}

void RequestQueue::enqueue(std::string id, std::string name, std::string docType,
                                std::string date, std::string reqs) {
    DocumentRequest* node = new DocumentRequest(id, name, docType, date, reqs);

    if (rear == nullptr) {
        front = rear = node;
    } else {
        rear->next = node;
```

```
        rear = node;
    }
    ++count;
}

DocumentRequest* RequestQueue::dequeue() {
    if (isEmpty()) return nullptr;

    DocumentRequest* node = front;
    front = front->next;

    if (front == nullptr) rear = nullptr;

    node->next = nullptr;
    --count;
    return node;
}

DocumentRequest* RequestQueue::peek() const {
    return front;
}

bool RequestQueue::isEmpty() const {
    return front == nullptr;
}

int RequestQueue::size() const {
    return count;
}

void RequestQueue::displayQueue() const {
    if (isEmpty()) {
        std::cout << "\nThere are no pending requests!\n";
        return;
    }
```

```
    std::cout << "\n======= PENDING REQUESTS =======\n";
    for (DocumentRequest* current = front; current != nullptr;
         current = current->next) {
        std::cout << "\nTicket Number : " << current->ticketID;
        std::cout << "\nResident Name: " << current->residentName;
        std::cout << "\nDocument Type: " << current->documentType;
        std::cout << "\nDate Submitted: " << current->dateSubmitted;
        std::cout << "\nRequirements  : " << current->requirements;
        std::cout << std::endl;
    }
}

bool convertStatus(const std::string& text, RequestStatus& status) {
    if (text == "1") status = RequestStatus::OnHold;
    else if (text == "2") status = RequestStatus::InProgress;
    else if (text == "3") status = RequestStatus::PendingApproval;
    else if (text == "4") status = RequestStatus::Ready;
    else if (text == "5") status = RequestStatus::Received;
    else return false;

    return true;
}

RequestStatus statusFromText(const std::string& text) {
    if (text == "On Hold") return RequestStatus::OnHold;
    if (text == "In Progress") return RequestStatus::InProgress;
    if (text == "Pending Approval") return RequestStatus::PendingApproval;
    if (text == "Ready") return RequestStatus::Ready;
    if (text == "Received") return RequestStatus::Received;
    return RequestStatus::OnHold;
}

int main() {
    RequestQueue requestQueue;
```

```
RequestList requestList;
DocumentList documentList;
Stack undoStack;
std::vector<DocReq*> processedRequests;

int ticketCounter = 1;
int choice;

do {
    std::cout << "\n========================================\n";
    std::cout << "       e-BARANGAY DOCUMENT SYSTEM\n";
    std::cout << "========================================\n";
    std::cout << "[1] Submit Document Request\n";
    std::cout << "[2] Process Next Request\n";
    std::cout << "[3] Update Request Status\n";
    std::cout << "[4] View All Request Records\n";
    std::cout << "[5] Search Request by Ticket Number\n";
    std::cout << "[6] View Pending Requests\n";
    std::cout << "[7] View Undo History\n";
    std::cout << "[8] Undo Last Action\n";
    std::cout << "[9] Sort Released Requests (Merge Sort)\n";
    std::cout << "[10] Exit\n";
    std::cout << "----------------------------------------\n";
    std::cout << "Enter your choice: ";

    if (!(std::cin >> choice)) {
        std::cout << "\nInvalid input. Please enter a number.\n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        continue;
    }
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    if (choice == 1) {
        std::string name, documentType, date, requirements;
```

```
        std::cout << "\n========== SUBMIT DOCUMENT REQUEST ==========\n";
        std::cout << "Resident Name: ";
        std::getline(std::cin, name);
        std::cout << "Document Type: ";
        std::getline(std::cin, documentType);
        std::cout << "Date Submitted (YYYY-MM-DD): ";
        std::getline(std::cin, date);
        std::cout << "Requirements: ";
        std::getline(std::cin, requirements);

        if (name.empty() || documentType.empty() ||
            date.empty() || requirements.empty()) {
            std::cout << "\nRequest not submitted. All fields are required.\n";
            continue;
        }

        std::string ticket = "BRG-2026-";
        if (ticketCounter < 10) ticket += "000";
        else if (ticketCounter < 100) ticket += "00";
        else if (ticketCounter < 1000) ticket += "0";
        ticket += std::to_string(ticketCounter++);

        requestQueue.enqueue(ticket, name, documentType, date, requirements);

        std::cout << "\nRequest successfully submitted!\n";
        std::cout << "Your Ticket Number: " << ticket << "\n";
        std::cout << "Your request has been added to the processing queue.\n";
    }

    else if (choice == 2) {
        DocumentRequest* nextRequest = requestQueue.dequeue();

        if (nextRequest == nullptr) {
            std::cout << "\nThere are no requests waiting to be processed.\n";
```

```
                continue;
            }

            RequestNode* record = requestList.addRequest(
                nextRequest->ticketID,
                nextRequest->documentType,
                nextRequest->residentName
            );

            if (record != nullptr) {
                record->addStatus(RequestStatus::OnHold,
                                  nextRequest->dateSubmitted, "Barangay Staff");

                std::cout << "\nRequest successfully moved for processing.\n";
                std::cout << "Ticket Number: " << nextRequest->ticketID << "\n";
                std::cout << "Resident Name: " << nextRequest->residentName << "\n";

                BarangayDocument document;
                document.specialId = nextRequest->ticketID;
                document.residentName = nextRequest->residentName;
                document.documentType = nextRequest->documentType;
                document.dateSubmitted = 0;
                document.fee = 0.0;
                documentList.insertSortedById(document);
            } else {
                std::cout << "\nUnable to create the request record.\n";
            }

            if (record != nullptr) {
                processedRequests.push_back(nextRequest);
            } else {
                delete nextRequest;
            }
        }
```

```
        else if (choice == 3) {
            std::string ticket, dateTime, staff;
            std::string oldStatus;
            RequestStatus newStatus;

            std::cout << "\n========== UPDATE REQUEST STATUS ==========\n";
            std::cout << "Enter Ticket Number: ";
            std::getline(std::cin, ticket);

            RequestNode* record = requestList.searchById(ticket);

            if (record == nullptr) {
                std::cout << "\nRequest not found. Process the request first.\n";
                continue;
            }

            if (record->historyTail != nullptr)
                oldStatus = statusToString(record->historyTail->status);
            else
                oldStatus = "On Hold";

            std::cout << "\nSelect New Status:\n";
            std::cout << "[1] On Hold\n";
            std::cout << "[2] In Progress\n";
            std::cout << "[3] Pending Approval\n";
            std::cout << "[4] Ready\n";
            std::cout << "[5] Received\n";
            std::cout << "Enter choice: ";

            std::string statusChoice;
            std::getline(std::cin, statusChoice);

            if (!convertStatus(statusChoice, newStatus)) {
                std::cout << "\nInvalid status selection.\n";
                continue;
```

```
            }

            std::cout << "Date and Time: ";
            std::getline(std::cin, dateTime);
            std::cout << "Staff Member: ";
            std::getline(std::cin, staff);

            if (dateTime.empty() || staff.empty()) {
                std::cout << "\nUpdate cancelled. Please complete all fields.\n";
                continue;
            }

            std::string updatedStatus = statusToString(newStatus);
            record->addStatus(newStatus, dateTime, staff);

            Action action;
            action.Action_name = "Update Request Status";
            action.Ticket_number = ticket;
            action.Previous_status = oldStatus;
            action.Updated_status = updatedStatus;
            undoStack.push(action);

            std::cout << "\nRequest status successfully updated!\n";
            std::cout << "Previous Status: " << oldStatus << "\n";
            std::cout << "Updated Status : " << updatedStatus << "\n";
        }

        else if (choice == 4) {
            std::cout << "\n========== ALL REQUEST RECORDS ==========\n";

            if (processedRequests.empty()) {
                std::cout << "No processed request records yet.\n";
            } else {
            requestList.printAll();
            }
```

```
        }

        else if (choice == 5) {
            std::string ticket;
            std::cout << "\n========== SEARCH REQUEST ==========\n";
            std::cout << "Enter Ticket Number: ";
            std::getline(std::cin, ticket);

            DocumentNode* result = documentList.FindById(ticket);

            if (result == nullptr) {
                std::cout << "\nNo matching document was found.\n";
            } else {
                std::cout << "\nDocument found!\n";
                std::cout << "Ticket Number: " << result->data.specialId << "\n";
                std::cout << "Resident Name: " << result->data.residentName << "\n";
                std::cout << "Document Type: " << result->data.documentType << "\n";
            }
        }

        else if (choice == 6) {
            requestQueue.displayQueue();
            std::cout << "\nTotal pending requests: " << requestQueue.size() << "\n";
        }

        else if (choice == 7) {
            std::cout << "\n========== UNDO HISTORY ==========\n";
            undoStack.display();
        }

        else if (choice == 8) {
            if (undoStack.isEmpty()) {
                std::cout << "\nThere are no actions available to undo.\n";
                continue;
            }
```

```
        Action lastAction = undoStack.pop();
        RequestNode* record =
            requestList.searchById(lastAction.Ticket_number);

        if (record != nullptr) {
            record->addStatus(statusFromText(lastAction.Previous_status),
                              "Undo operation", "Barangay Staff");

            std::cout << "\nLast action successfully undone!\n";
            std::cout << "Ticket Number: " << lastAction.Ticket_number << "\n";
            std::cout << "Restored Status: " << lastAction.Previous_status << "\n";
        } else {
            std::cout << "\nThe related request record could not be found.\n";
        }
    }

    else if (choice == 9) {
        if (processedRequests.empty()) {
            std::cout << "\nThere are no released requests to sort yet.\n";
            continue;
        }

        std::cout << "\n========== SORT RELEASED REQUESTS ==========";
        std::cout << "\n[1] Sort by Date Submitted";
        std::cout << "\n[2] Sort by Document Type";
        std::cout << "\n[3] Sort by Ticket Number";
        std::cout << "\nEnter sorting option: ";

        int sortChoice;
        if (!(std::cin >> sortChoice)) {
            std::cout << "\nInvalid input. Please enter a number.\n";
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            continue;
```

```
            }
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

            // Sort a copy so the original processed-request collection is retained.
            std::vector<DocReq*> sortedRequests = processedRequests;

            if (sortChoice == 1) {
                mergeSort(sortedRequests, byDate);
                std::cout << "\n--- RELEASED REQUESTS SORTED BY DATE ---\n";
            } else if (sortChoice == 2) {
                mergeSort(sortedRequests, byDocType);
                std::cout << "\n--- RELEASED REQUESTS SORTED BY DOCUMENT TYPE ---\n";
            } else if (sortChoice == 3) {
                mergeSort(sortedRequests, byTicketID);
                std::cout << "\n--- RELEASED REQUESTS SORTED BY TICKET NUMBER ---\n";
            } else {
                std::cout << "\nInvalid sorting option. Please select from 1 to 3.\n";
                continue;
            }

            std::cout << "\n";
            for (const DocReq* request : sortedRequests) {
                std::cout << "Ticket Number : " << request->ticketID;
                std::cout << "\nResident Name: " << request->residentName;
                std::cout << "\nDocument Type: " << request->documentType;
                std::cout << "\nDate Submitted: " << request->dateSubmitted;
                std::cout << "\n----------------------------------------\n";
            }
        }

        else if (choice == 10) {
            std::cout << "\nThank you for using e-BarangayDocs!\n";
        }

        else {
```

```
            std::cout << "\nInvalid choice. Please select from 1 to 10.\n";
        }

    } while (choice != 10);

    // Delete processed request nodes after the program finishes.
    for (DocReq* request : processedRequests) {
        delete request;
    }

    return 0;
}
```

A.2 BINARY_SEARCH.h

```
#ifndef BINARY_SEARCH_H
#define BINARY_SEARCH_H

#include <cstddef>
#include <string>
#include <utility>

struct BarangayDocument {
    std::string specialId;
    std::string residentName;
    std::string documentType;
    int dateSubmitted;
    double fee;
};

struct DocumentNode {
    BarangayDocument data;
    DocumentNode* next;
    explicit DocumentNode(const BarangayDocument& doc) : data(doc), next(nullptr) {}
};
```

```
class DocumentList {
private:
    DocumentNode* headPtr;
    DocumentNode* tailPtr;
    std::size_t count;

    static DocumentNode* middle(DocumentNode* start, DocumentNode* end) {
        DocumentNode* slow = start;
        DocumentNode* fast = start;
        while (fast != end && fast->next != end) {
            slow = slow->next;
            fast = fast->next->next;
        }
        return slow;
    }

public:
    DocumentList() : headPtr(nullptr), tailPtr(nullptr), count(0) {}
    ~DocumentList() {
        while (headPtr != nullptr) {
            DocumentNode* temp = headPtr;
            headPtr = headPtr->next;
            delete temp;
        }
    }

    DocumentList(const DocumentList&) = delete;
    DocumentList& operator=(const DocumentList&) = delete;
    DocumentNode* head() const { return headPtr; }
    std::size_t size() const { return count; }
    bool empty() const { return count == 0; }



    void append(const BarangayDocument& doc) {
        DocumentNode* node = new DocumentNode(doc);
```

```
        if (tailPtr == nullptr) {
            headPtr = tailPtr = node;
        } else {
            tailPtr->next = node;
            tailPtr = node;
        }
        ++count;
    }

    void insertSortedById(const BarangayDocument& doc) {
        DocumentNode* node = new DocumentNode(doc);

        if (headPtr == nullptr || doc.specialId < headPtr->data.specialId) {
            node->next = headPtr;
            headPtr = node;
            if (tailPtr == nullptr) tailPtr = node;
        } else {
            DocumentNode* cur = headPtr;
            while (cur->next != nullptr &&
                   cur->next->data.specialId < doc.specialId) {
                cur = cur->next;
            }
            node->next = cur->next;
            cur->next = node;
            if (node->next == nullptr) tailPtr = node;
        }
        ++count;
    }

    bool isSortedById() const {
        for (DocumentNode* cur = headPtr; cur && cur->next; cur = cur->next) {
            if (cur->data.specialId > cur->next->data.specialId) return false;
        }
        return true;
    }
```

```
bool isSortedByDate() const {
    for (DocumentNode* cur = headPtr; cur && cur->next; cur = cur->next) {
        if (cur->data.dateSubmitted > cur->next->data.dateSubmitted) return false;
    }
    return true;
}

DocumentNode* FindById(const std::string& targetId) const {
    DocumentNode* start = headPtr;
    DocumentNode* end = nullptr;

    while (start != end) {
        DocumentNode* mid = middle(start, end);
        int cmp = mid->data.specialId.compare(targetId);
        if (cmp == 0) return mid;
        else if (cmp < 0) start = mid->next;
        else end = mid;
    }
    return nullptr;
}

DocumentNode* LowerBoundByDate(int date) const {
    DocumentNode* start = headPtr;
    DocumentNode* end = nullptr;

    while (start != end) {
        DocumentNode* mid = middle(start, end);
        if (mid->data.dateSubmitted < date) start = mid->next;
        else end = mid;
    }
    return start;
}

DocumentNode* UpperBoundByDate(int date) const {
```

```
        DocumentNode* start = headPtr;
        DocumentNode* end = nullptr;

        while (start != end) {
            DocumentNode* mid = middle(start, end);
            if (mid->data.dateSubmitted <= date) start = mid->next;
            else end = mid;
        }
        return start;
    }

    std::pair<DocumentNode*, DocumentNode*> findDateRange(int fromDate, int toDate) const {
        if (fromDate > toDate) return std::make_pair(nullptr, nullptr);
        return std::make_pair(LowerBoundByDate(fromDate), UpperBoundByDate(toDate));
    }

    static std::size_t CountRange(const DocumentNode* first, const DocumentNode* last) {
        std::size_t total = 0;
        for (const DocumentNode* p = first; p != last; p = p->next) ++total;
        return total;
    }
};

#endif
```

A.3 LinkedList.h

```
#ifndef LINKEDLIST_H
#define LINKEDLIST_H

#include <cctype>
#include <iomanip>
#include <iostream>
#include <string>
```

```
enum class RequestStatus {
    OnHold, //in queue/waiting
    InProgress,
    PendingApproval, //by the captain
    Ready, //when approved
    Received
};



inline const char *statusToString(RequestStatus s) {
    switch (s) {
        case RequestStatus::OnHold:          return "On Hold";
        case RequestStatus::InProgress:      return "In Progress";
        case RequestStatus::PendingApproval: return "Pending Approval";
        case RequestStatus::Ready:           return "Ready";
        case RequestStatus::Received:        return "Received";
    }
    return "Unknown";
}

struct StatusNode {
    RequestStatus status;
    std::string dateTime;
    std::string staff;
    StatusNode *next;

    StatusNode(RequestStatus s, const std::string &dt, const std::string &st)
        : status(s), dateTime(dt), staff(st), next(nullptr) {}
};

struct RequestNode {
    std::string requestId;
    std::string documentType;
    std::string requesterName;
```

```
    StatusNode *historyHead;
    StatusNode *historyTail;
    RequestNode *next;

    RequestNode(const std::string &id, const std::string &type, const std::string &requester)
        : requestId(id), documentType(type), requesterName(requester),
          historyHead(nullptr), historyTail(nullptr), next(nullptr) {}

    ~RequestNode() {
        while (historyHead) {
            StatusNode *n = historyHead->next;
            delete historyHead;
            historyHead = n;
        }
    }

    RequestNode(const RequestNode &) = delete;
    RequestNode &operator=(const RequestNode &) = delete;


    void addStatus(RequestStatus status, const std::string &dateTime,
                   const std::string &staff) {
        StatusNode *node = new StatusNode(status, dateTime, staff);
        if (historyTail) historyTail->next = node;
        else             historyHead = node;
        historyTail = node;
    }
};


class RequestList {
private:
    RequestNode *head;
    RequestNode *tail;
```

```
public:
    RequestList() : head(nullptr), tail(nullptr) {}

    RequestList(const RequestList &) = delete;
    RequestList &operator=(const RequestList &) = delete;

    ~RequestList() {
        while (head) {
            RequestNode *n = head->next;
            delete head;
            head = n;
        }
    }

    // Checks ID if format is "BRG-YYYY-****"
    static bool isValidRequestId(const std::string &id) {
        if (id.size() != 13) return false;
        if (id.compare(0, 4, "BRG-") != 0) return false;
        if (id[8] != '-') return false;
        for (int i = 4; i < 8; ++i)
            if (!std::isdigit(static_cast<unsigned char>(id[i]))) return false;
        for (int i = 9; i < 13; ++i)
            if (!std::isdigit(static_cast<unsigned char>(id[i]))) return false;
        return true;
    }

    RequestNode *addRequest(const std::string &requestId,
                            const std::string &documentType,
                            const std::string &requesterName) {
        if (!isValidRequestId(requestId)) return nullptr;
        RequestNode *node = new RequestNode(requestId, documentType, requesterName);
        if (tail) tail->next = node;
        else      head = node;
        tail = node;
```

```
        return node;
    }

    RequestNode *searchById(const std::string &requestId) const {
        for (RequestNode *r = head; r; r = r->next)
            if (r->requestId == requestId) return r;
        return nullptr;
    }

    void printAll() const {
        for (const RequestNode *r = head; r; r = r->next) {
            std::cout << "Request " << r->requestId << " | " << r->documentType
                      << " | " << r->requesterName << "\n";
            int i = 1;
            for (const StatusNode *s = r->historyHead; s; s = s->next, ++i) {
                std::cout << "  " << i << ". " << std::left
                          << std::setw(18) << statusToString(s->status)
                          << std::setw(22) << s->dateTime
                          << s->staff << "\n";
            }
            std::cout << "\n";
        }
    }
};

#endif
```

A.4 MergeSort.h

```
#ifndef MERGESORT_H
#define MERGESORT_H

#include <vector>
#include <string>
#include "Queue.h"
```

```
using DocReq = DocumentRequest;

inline bool byDate(const DocReq* a, const DocReq* b) {
    return a->dateSubmitted < b->dateSubmitted;
}

inline bool byDocType(const DocReq* a, const DocReq* b) {
    return a->documentType < b->documentType;
}

inline bool byTicketID(const DocReq* a, const DocReq* b) {
    return a->ticketID < b->ticketID;
}

template <typename Compare>
void merge(std::vector<DocReq*>& arr, int left, int mid, int right, Compare comp) {
    std::vector<DocReq*> temp;
    temp.reserve(right - left + 1);

    int i = left;
    int j = mid + 1;

    while (i <= mid && j <= right) {
        if (comp(arr[j], arr[i])) temp.push_back(arr[j++]);
        else                      temp.push_back(arr[i++]);
    }
    while (i <= mid)   temp.push_back(arr[i++]);
    while (j <= right) temp.push_back(arr[j++]);

    for (int k = 0; k < static_cast<int>(temp.size()); k++) {
        arr[left + k] = temp[k];
    }
}
```

```
template <typename Compare>
void mergeSortHelper(std::vector<DocReq*>& arr, int left, int right, Compare comp) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;
    mergeSortHelper(arr, left, mid, comp);
    mergeSortHelper(arr, mid + 1, right, comp);
    merge(arr, left, mid, right, comp);
}

template <typename Compare>
void mergeSort(std::vector<DocReq*>& arr, Compare comp) {
    if (arr.size() < 2) return;
    mergeSortHelper(arr, 0, static_cast<int>(arr.size()) - 1, comp);
}

#endif
```

A.5 Queue.cpp

```
#include "Queue.h"

RequestQueue::~RequestQueue() {
    while (!isEmpty()) {
        dequeue();
    }
}

void RequestQueue::enqueue(std::string id, std::string name, std::string docType, std::string
date, std::string reqs) {
    DocumentRequest* newRequest = new DocumentRequest(id, name, docType, date, reqs);
    if (isEmpty()) {
        front = rear = newRequest;
    } else {
        rear->next = newRequest;
```

```
        rear = newRequest;
    }
    count++;
}

DocumentRequest* RequestQueue::dequeue() {
    if (isEmpty()) return nullptr;
    DocumentRequest* temp = front;
    front = front->next;
    if (front == nullptr) rear = nullptr;
    count--;
    return temp;
}

DocumentRequest* RequestQueue::peek() const {
    return front;
}

bool RequestQueue::isEmpty() const {
    return front == nullptr;
}

int RequestQueue::size() const {
    return count;
}

void RequestQueue::displayQueue() const {
    if (isEmpty()) {
        std::cout << "Queue is empty.\n";
        return;
    }
    DocumentRequest* curr = front;
    std::cout << "\n--- Current Pending Processing Queue ---\n";
    while (curr != nullptr) {
        std::cout << "[" << curr->ticketID << "] " << curr->residentName
```

```
                    << " - " << curr->documentType << " (" << curr->dateSubmitted << ")\n";
        curr = curr->next;
    }
}
```

A.6 Queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#include <string>
#include <iostream>

struct DocumentRequest {
    std::string ticketID;
    std::string residentName;
    std::string documentType;
    std::string dateSubmitted;
    std::string requirements;
    DocumentRequest* next;

    DocumentRequest(std::string id, std::string name, std::string docType, std::string date,
std::string reqs)
        : ticketID(id), residentName(name), documentType(docType), dateSubmitted(date),
requirements(reqs), next(nullptr) {}
};

class RequestQueue {
private:
    DocumentRequest* front;
    DocumentRequest* rear;
    int count;

public:
    RequestQueue() : front(nullptr), rear(nullptr), count(0) {}
    ~RequestQueue();
```

```
    void enqueue(std::string id, std::string name, std::string docType, std::string date,
std::string reqs);
    DocumentRequest* dequeue();
    DocumentRequest* peek() const;
    bool isEmpty() const;
    int size() const;
    void displayQueue() const;
};

#endif
```

A.7 stack.h

```
#ifndef STACK_H
#define STACK_H
#include <string>
#include <iostream>

struct Action {
    std::string Action_name;
    std::string Ticket_number;
    std::string Previous_status;
    std::string Updated_status;
};

class Stack {
    private:
    static const int MAX = 100;
    Action arr[MAX];
    int top;

    public:
    Stack() {
        top = -1;
```

```
}

bool isEmpty() {
    return top == -1;
}

bool isFull() {
    return top == MAX - 1;
}

void push(Action action) {
    if (isFull()) {
        std::cout << "The stack is full!\n";
        return;
    }
    arr[++top] = action;
    std::cout << "This action has been successfully added!\n";
}

Action pop() {
    if (isEmpty()) {
        std::cout << "There is no actions available to undo!\n";
        return {"", "", "", ""};
    }
    return arr[top--];
}

void peek() {
    if (isEmpty()) {
        std::cout << "The stack is empty!\n";
        return;
    }
    std::cout << "\nLatest Action: " << arr[top].Action_name;
    std::cout << "\nTicket Number: " << arr[top].Ticket_number;
    std::cout << "\nPrevious Status: " << arr[top].Previous_status;
```

```
        std::cout << "\nUpdated Status: " << arr[top].Updated_status;
        std::cout << std::endl;
    }

    void display() {
        if (isEmpty()) {
            std::cout << "there are no recorded actions!\n";
            return;
        }
        std::cout << "\n======= AVAILABLE UNDO ACTIONS =======\n";

        for (int i = top; i >= 0; i--) {
            std::cout << "\nAction Taken: " << arr[i].Action_name;
            std::cout << "\nTicket Number: " << arr[i].Ticket_number;
            std::cout << "\nPrevious Status: " << arr[i].Previous_status;
            std::cout << "\nUpdated Status: " << arr[i].Updated_status;
            std::cout << std::endl;
        }
    }
};

#endif
```

Appendix B. Team Composition

#	Name (Surname, First Name M.I.)	Student No.	Role	Main Contributions	%
1	Antonio, Cristine Mae	2515041	Linked List Developer, Presentation Designer	Linked List Module, PPT, Documentation	100
2	Bungag, Marc Jhowell	2514152	Code Implementation Lead	Binary Search Module, Main Module, Documentation	100
3	Limpio, Ramelissa Melith	2515925	Merge Sort Developer, Presentation Designer	Merge module, PPT	100

4	Macatangay, Justin Jake	2514647	Flowchart Designer	Flowchart,	100
5	Mendoza, Allyson	2510040	Stack Developer	Stack Module, Documentation	100
6	Silao, Johrel Louie	2513830	Queue Developer / Repository Manager	Queue Module, Project Documentation, GitHub Repository Design	100

Appendix C. GitHub Repository

Item	Details
Repository link	https://github.com/Limpiocpe/CPE010_FinalProj_Team1
Branch / release submitted	Code-implementation branch (https://github.com/Limpiocpe/CPE010_FinalProj_Team1/tree/Code-Implementation)
README contents	Team Members, Project Description, Project Documentation, Video Presentation, Code Implementation
How to compile	g++ -std=c++17 -O2 main.cpp -o system

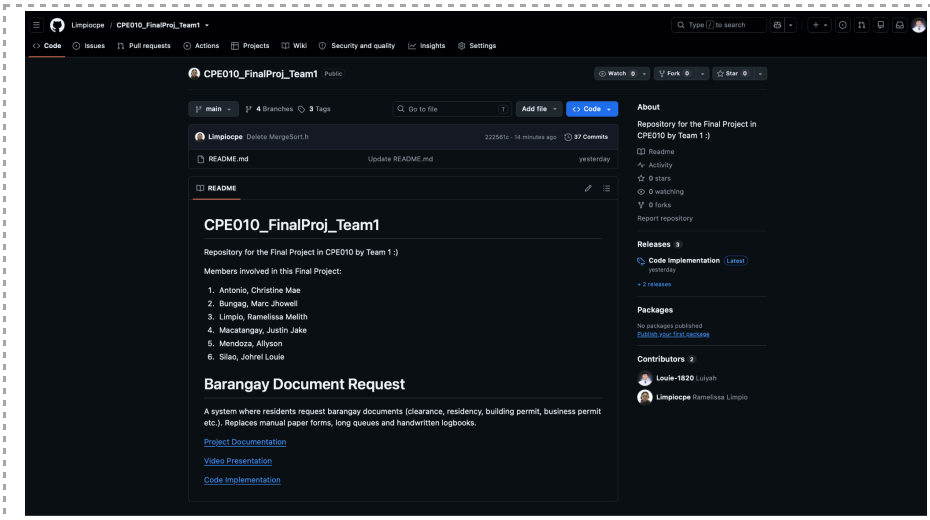


Figure 6. Screenshot of the repository page showing files and commit history

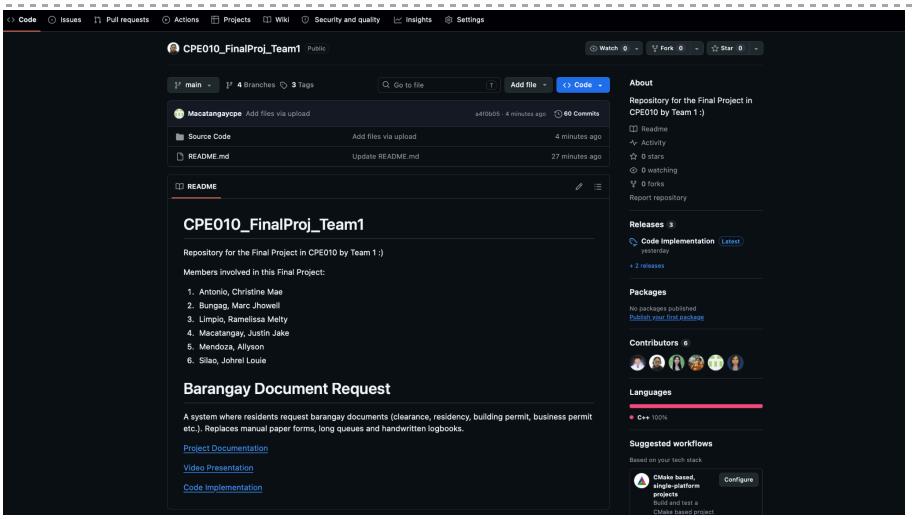


Figure 7. Screenshot of the repository page showing files and commit history

EVALUATION CRITERIA

Criteria	Description	Points
Business Process	Realistic process clearly described; problems and proposed system well defined	15
Data Structures and Algorithms Applied	At least five course topics, correctly explained and justified for the business process	25
System Flowchart	Complete, correct symbols, matches the program	10
Output	All features shown working, including invalid input handling	15
Performance Analysis	Correct theoretical complexity; sound experimental method; meaningful comparison	25
Documentation and Appendices	Complete code, team composition, working GitHub repository; clear writing	10
Total		100